CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

NAAC GRADE A+ ACCREDITED UNIVERSITY

QS UNIVERSITY RANKINGS ASIA 2025
#1 AMONG ALL PRIVATE UNIVERSITIES IN INDIA

OBJECT-ORIENTED PROGRAMMING / CASE STUDY

ROLE OF EXCEPTION HANDLING IN RELIABLE SOFTWARE SYSTEMS

A practical study of safer failures, clearer error handling, and resource-aware C++ design.

SUBMITTED BY	Sania Amjad
UID / STUDENT ID	25BCD10080
PROGRAMME	BCA Data Science
INSTITUTION	Chandigarh University
SUBJECT	C++ / Object-Oriented Programming
ACADEMIC SESSION	2025-2028

Prepared as an academic case study on the use of C++ exception handling to make programs safer, more predictable, and easier to maintain.

Abstract & Study Framework

Reliable software should respond to expected inputerrors andunexpected operating conditions without crashing unpredictably or silently corrupting data. In C++, exception handling provides a structured mechanism for reporting and handling failures. The three central keywords are **try**, **throw**, and **catch**.

This case study examines exception handling in three common situations: invalid user input, file-operation failures, and runtime or domain errors. It explains how exceptions travel through function calls, how standard exception classes can be used, and how resource-management practices such as RAII help keep files and memory safe.

CORE MESSAGE

Exceptions can improve reliability, but they do not automatically make software correct. Validation, clear messages, testing, and recovery design remain necessary.

Objectives

- Explain thepurpose and syntax of try, throw, and catch in C++.
- Identify situations in which exception handling is useful.
- Demonstrate handling of invalid input, file errors, and domain-specific failures.
- Differentiate exceptions from ordinary conditional checks and error codes.
- Discuss exception safety, resource management, testing, and design limitations.

Scope and method

The case study uses a conceptual review of C++ language features and illustrative code examples. The examples are educational fragments intended to demonstrate design choices; they are not a complete production application.

Reliability goals

Goal	What it means
Predictability	The application responds consistently to failure conditions.
Integrity	Failed operations do not leave data in an incorrect state.
Usability	Users receive understandable messages instead of cryptic crashes.
Maintainability	Error-handling logic is organized rather than scattered.

01 / Introduction to Software Reliability

Software reliability is the ability of a system to performits requiredfunctionsconsistently under stated conditions. A reliable application does not merely work for ideal inputs. It also handles invalid values, missing files, unavailable resources, and other failures in a controlled way.

Consider a library application that asks a user to enter a book ID and then writes an issue record to a file. The user may enter letters instead of a number, the requested book may not exist, or the output file may fail to open. If these situations are ignored, the application may display a misleading success message, stop unexpectedly, or lose important information.

Exception handling separates the code that detects a failure from the code that decides how to respond. A low-level function can report a problem, while a higher-level function decides whether to retry, display a message, cancel the operation, or terminate safely.

Key terms

Term	Meaning
Failure	An operation cannot complete as intended, such as opening a missing file.
Exception	An object or value used to signal an exceptional condition.
Throwing	Transferring control by reporting an exception with throw.
Catching	Handling a matching exception in a catch block.
Recovery	Taking a safe action after a failure, such as retrying or cancelling.

Detect failure → **Report exception** → **Handle at suitable level** → **Recover safely**

DESIGN PRINCIPLE

Do not claim an operation succeeded until its important steps have actually succeeded.

02 / How try, throw, and catch Work

C++exception handling normally involves three stages. Thetry blockcontains statements that may result in an exception. The throw expression signals the failure and transfers control away from the current execution path. A compatible catch handler receives the exception and executes a response.

try → throw → catch → continue safely

```
#include  <iostream>
#include <stdexcept>
using namespace std;

intmain(){
    try{
        intage = -2;
        if(age < 0)
            throw invalid_argument("Age cannot be negative");
        cout<< "Age accepted";
    }
    catch(const invalid_argument& ex) {
        cout<< "Input error: " << ex.what();
    }
}
```

The program detects an invalid value and throws an invalid_argument exception. The matching catch block prints a clear message. The statement after throw in the try block is not executed. The program continues after the catch block if the exception is handled there.

Important language elements

- **try:**marksaregion wherean exception may be thrown.
- **throw:** signals an exception and begins stack unwinding.
- **catch:** declares a handler for a matching exception type.
- **what():** returns a human-readable description for standard exception objects.

Catch matching and propagation

A handler must match the exception type. More specific handlers should normally appear before broader handlers. If a function does not handle an exception locally, it can propagate to its caller. C++ searches outward for a matching handler; during this process, local automatic objects are destroyed. This is known as **stack unwinding**. If no matching handler is found, the program calls std::terminate.

03 / Case One: Handling Invalid User Input

Invalid input is one of the most frequent causes ofunexpectedprogrambehaviour.Examples include a negative quantity,

an empty required name, an unsupported menu option, or a value outside an accepted range. Input validation should happen close to where data enters the program.

For simple menu choices, an if statement may be enough. Exceptions are useful when a function cannot perform its contract and needs to report the problem to a caller that can decide what to do.

```
#include <stdexcept>
using namespace std;
double calculateTotal(double price, int quantity) {

if (price < 0 || quantity <= 0) {
throw invalid_argument("Price or quantity is invalid");

}
return price * quantity;
}
```

A caller can wrap the function call in try/catch and ask the user to enter the values again. The function itself does not need to know how the user interface works.

Input handling recommendations

- Checknumericrangesand requiredfields before performing calculations.
- Use clear messages that explain what the user can correct.
- Do not use exceptions for every ordinary menu choice; use simple conditions for expected branching.
- When reading from cin, check stream state because entering text where a number is expected can set a failure flag.

```
int quantity;
cout << "Enter quantity: ";
if (!(cin >> quantity)) {

cout << "Please enter a whole number.\n";
cin.clear();
cin.ignore(10000, '\n');
```

IMPORTANT DISTINCTION

Stream-state checking is not exception handling, but it is a valid way to validate input. Exceptions and ordinary validation complement one another.

04 / Case Two: File-Operation Errors

Applicationsrely on files forconfiguration, reports, transaction histories, or saved records. A file may fail to open because the path is incorrect, the directory does not exist, permission is denied, or storage is unavailable. A reliable program checks whether file operations succeeded before claiming that data was saved.

```
#include <fstream>
#include <stdexcept>
#include <string>
using namespace std;

void saveRecord(const string& path) {
    ofstream file(path, ios::app);
    if (!file.is_open())
        throw runtime_error("Unable to open output file");
    file << "Record saved" << '\n';
    if (!file)
        throw runtime_error("Writing the record failed");
}
```

The calling code can catch runtime_error and display a message such as: “The record could not be saved. Check the file path and permissions.” In a real application, the program should avoid reporting success until the write has been checked.

Connection to a library management program

A library system might append a book ID and borrower name to issued_books.txt. If the output stream fails to open, the issue record should not be treated as safely persisted. The application should decide whether to cancel the issue operation or mark it as pending. Changing in-memory availability without successfully recording the issue can create inconsistent data.

Resource safety: RAII

Create file object	→	**Use resource**	→	**Object leaves scope**	→	**Destructor closes file**

The file object is an automatic object. When it goes out of scope, its destructor closes the file. This illustrates **RAII (Resource Acquisition Is Initialization)**, which ties resource lifetime to object lifetime and reduces the need for manual cleanup on every error path.

05 / Case Three: Runtime and Domain Errors

Someerrors arise not from invalid syntax butfromvaluesorstates thatmake anoperationimpossible. Examples include dividing by zero in a business calculation, requesting a nonexistent account, withdrawing more than the permitted balance, or processing an order with insufficient stock.

The program should distinguish between a programming defect and a condition that the application can reasonably report. Exceptions can express a failed operation, but they do not replace sound logic or testing.

Example: a safe division function

```
#include <stdexcept>
using namespace std;

double divide(double numerator, double denominator) {
    if (denominator == 0.0) {
        throw domain_error("Denominator must not be zero");
    }
    return numerator / denominator;
}
```

The domain_error type communicates that the requested operation is not valid for the supplied values. The caller may catch it and ask for another value. More complicated calculations may also need checks for overflow, precision, or domain-specific limits.

Standard exception categories

Exception type	Typical use
invalid_argument	A function receives an unacceptable argument.
out_of_range	An index or value falls outside an allowed range.
runtime_error	A runtime operation fails for a reason represented by the application.
logic_error	A problem indicates logical misuse or a violated contract.
domain_error	An argument is outside the mathematical domain of an operation.

These exception types are declared in the standard **<stdexcept>** header. Choosing a meaningful type makes errors easier to interpret and catch selectively.

06 / Exception Propagation and Program Design

Anexception doesnothave tobecaught inthesamefunctioninwhich it isthrown. Often,alower-level function detects a problem while a higher-level function has enough context to decide the response. This separation can make a program easier to maintain.

```
void issueBook(int bookId) {
    // Search for the book...
    if (bookId <= 0) {
        throw invalid_argument("Book ID must be positive");
    }
    // Continue with issue logic...
}

intmain(){
    try{
        issueBook(-10);
    }
    catch(const exception& ex) {
        cerr<< "Operation failed: " << ex.what() << '\n';
    }
}
```

Here, issueBook() detects an invalid ID and throws an exception. main() catches the standard exception through a reference to const exception. Catching by const reference avoids copying the exception object and preserves polymorphic behaviour.

Where should an exception be caught?

- Catchan exception wherethe program cantakeauseful action.
- Translate low-level technical errors into user-friendly messages at the user-interface boundary.
- Do not catch an exception only to ignore it.
- Avoid overly broad handlers if they prevent important errors from being handled correctly.
- Keep error messages useful without exposing passwords, private records, or sensitive internal details.

Exceptions versus ordinary control flow

Exceptions are intended for exceptional or failed operations, not as a replacement for every if/else statement. If a condition is routine and expected, such as checking whether a menu choice equals 1, direct conditional logic is usually clearer.

> **RULE OF THUMB**
> Use conditions for normal decisions. Use exceptions when an operation cannot fulfil its contract and the caller needs to be informed.

07 / Exception Safety and Memory Management

Anexceptioncan interrupt a function at almost any point where an exception ispermitted. If a programmanually allocates memory or acquires a resource and then throws before releasing it, the resource may leak. C++ addresses this problem through RAII and standard library resource-management types.

Unsafe manual resource handling

```
void example() {
    int* values = new int[100];
    // An operation here might throw.
    // If it throws, delete[] may never execute.
    delete[] values;
}
```

If an exception leaves the function before delete[] is reached, the allocated array may not be released. A modern alternative is to use a standard container or smart pointer that manages the allocation automatically.

Safer alternative using vector

```
#include <vector>
using namespace std;

void example() {
    vector<int> values(100);
    // If an exception occurs, values is destroyed
    // automatically and its memory is released.
}
```

The vector manages its own memory. When stack unwinding destroys the vector object, its destructor releases the allocated storage. For a single dynamically allocated object, std::unique_ptr is often appropriate; use std::shared_ptr only when shared ownership is genuinely needed.

Exception-safety guarantees

Guarantee	Meaning
No-throw	The operation promises not to let exceptions escape.
Strong	If the operation fails, observable program state remains unchanged.
Basic	If the operation fails, invariants remain valid and resources are not leaked.
No guarantee	State may be invalid or resources may be lost; this is undesirable.

08 / Comparing Error-Handling Methods

C++programscan reportfailuresthrough exceptions, returncodes,status objects, orsimple conditions. The most suitable approach depends on whether the condition is expected, how far the error must travel, and whether callers need detailed information.

Method	Advantages	Limitations / suitable use
if / else	Simple and readable for routine conditions.	Can become repetitive if many nested checks are required.
Return codes	Explicit and often efficient; caller decides what to do.	Every caller must check the result consistently.
Exceptions	Separate failure paths from normal logic; propagate through calls.	Can complicate control flow if used carelessly.
Status/result object	Can carry success or failure data explicitly.	Requires a consistent result type and explicit checking.

Illustrative return-code approach

```
bool saveRecord(const string& path) {
    ofstream file(path, ios::app);
    if (!file) return false;
    file << "Record saved\n";
    return static_cast<bool>(file);
}
```

This function returns true when the write succeeds and false otherwise. The caller must check the result. It can be a good design for a small, expected operation failure.

Illustrative exception-based approach

```
void saveRecordOrThrow(const string& path) {
    ofstream file(path, ios::app);
    if (!file) throw runtime_error("Could not open file");
    file << "Record saved\n";
    if (!file) throw runtime_error("Could not write file");
}
```

COMPARISON TAKEAWAY

Neither approach is automatically superior. Consistency, clear contracts, and reliable handling by callers are essential.

09 / Testing Strategy and Best Practices

Exception handling shouldbe verifiedwith teststhatdeliberately triggerfailureconditions.Testing only the successful path does not show whether the program handles invalid inputs or resource failures safely.

Test	Input / condition	Expected behaviour
Valid input	Positive quantity and valid price	Calculation completes normally.
Invalid range	Quantity is zero or negative	invalid_argument is thrown and handled.
Missing output path	Directory does not exist	File-opening failure is reported; no false success.
Write failure	Simulated or induced output-stream failure	Program detects the failed write.
Unhandled exception	Throw without a matching handler in a controlled test	Uncaught exception leads to termination; not for production.
Resource cleanup	Throw while a vector or file object is in scope	Automatic objects are destroyed during stack unwinding.

Best practices

- Throw meaningfulstandard exception types or a well-designed custom exception.
- Catch by const reference when handling polymorphic standard exceptions.
- Keep try blocks focused so it is clear which operation may fail.
- Use RAII, containers, and smart pointers to manage resources safely.
- Do not use exceptions to silently discard failures or control routine loops.
- Test both success and failure paths, including file and resource failures.
- Log enough context for diagnosis while avoiding exposure of sensitive information.

Limitations

Exceptions cannot guarantee that a failed operation is recoverable. A hardware failure, process termination, or power loss may prevent cleanup or saving. File writes may also require stronger durability measures for critical systems. Good exception handling reduces avoidable failures but must be combined with validation, testing, and appropriate system design.

10 / Conclusion & References

Exception handling is an important C++ feature for building softwarethat responds to failures in a structured and

understandable way. The try block identifies the region in which failures may be reported, throw signals an exceptional condition, and catch provides a place to handle a matching exception. Together, these features allow low-level operations to report failures while higher-level code decides how the application should respond.

The examples show how exceptions can help validate function arguments, report file-operation failures, and handle invalid domain operations. They also demonstrate that exceptions must be designed carefully. Routine conditions are often better handled with ordinary if statements or return values, while exceptions are useful when an operation cannot complete and the caller needs to know why.

A key conclusion is that exception handling and memory management are connected. During stack unwinding, automatic objects are destroyed. RAII, standard containers, and smart pointers use this behaviour to release resources reliably. Modern C++ design should therefore prefer automatic resource management over scattered manual cleanup.

FINAL TAKEAWAY

Exception handling improves reliability when paired with meaningful messages, suitable catch locations, input validation, resource safety, and systematic testing. It is one tool in robust software design—not a substitute for good design.

References

• Bjarne Stroustrup,The C++ Programming Language, 4th Edition, Addison-Wesley, 2013.

• Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo, C++ Primer, 5th Edition, Addison-Wesley, 2012.

• cppreference.com, “Exceptions” — https://en.cppreference.com/w/cpp/language/exceptions

• cppreference.com, “std::exception” — https://en.cppreference.com/w/cpp/error/exception

• cppreference.com, “std::vector” — https://en.cppreference.com/w/cpp/container/vector

• ISO C++ working draft, sections covering exception handling and library exception types — https://www.open-std.org/jtc1/sc22/wg21/